GPU Architecture

BE BOUNDLESS

Ice breaker

I'm so sorry y'all but this helps me I swear

> **Please tell me:**
  - **Name**
  - **Pronouns**
  - **Year at UW**
  - **Favorite potato dish**

NVIDIA

ASIC Hardware Design Engineer - New College Grad 2026

Austin, TX · 2 weeks ago · Over 100 people clicked apply
Promoted by hirer · Responses managed off LinkedIn

$116K/yr - $218.5K/yr | Full-time

NVIDIA Corp

NASDAQ: NVDA

4.62 trillion USD

Market capitalization

Why this project

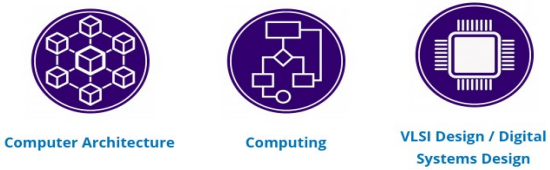
"GPUs are pretty cool" is my personal reason, but if you're not sold off that (disclaimer this is my opinion)

> **Chip market is lots of money**
  - **Right now massive thanks to AI**
> **Takes a while to learn chip design at uw**
  - **Path is multiple years before the chance for exploration.**
  - **CSE 12X → EE 271/CSE 369 → EE/CSE 469**
  - **+ time waiting for your standing**

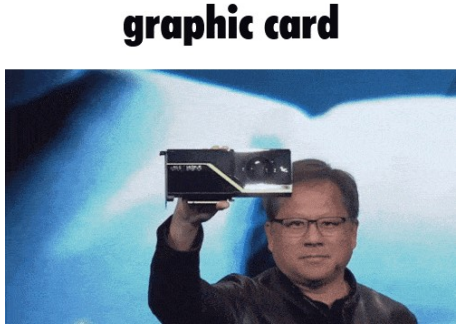
Why this project

So I'm not saying this project will get you a job at NVIDIA/AMD

- **But this project is a good chance to practice/learn/demonstrate interest and capability in some real hardware design skills as used in industry**
- **And at least three pathways in the ECE major (not sure abt CE my bad)**



Basics



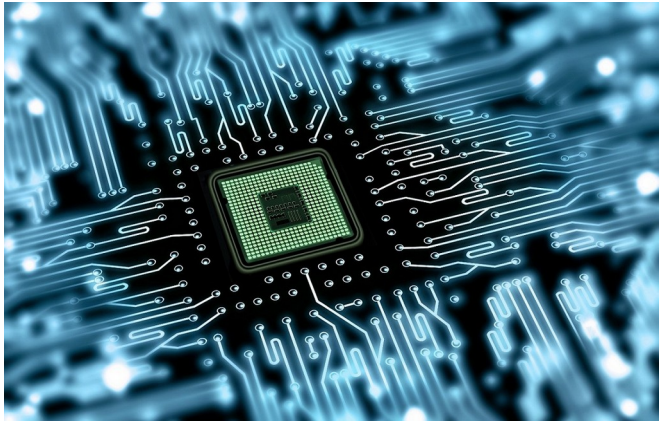
What the hell is a GPU – lets peel back the layers

> **A Graphics Processing Unit is a type of processor that handles your computers graphics**
> **Actually really good at doing lots of math in parallel**
  - **Big deal for machine learning**
> **They are considered a kind of <u>processor</u>**

Basics

What is a processor?

- **Specialized circuit that can turn inputs (*instructions)* into actions (math, load, store, branch)**
- **These can be designed to accomplish our needs, in this case, matrix math**



Basics

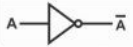
How do we make rocks do this math?

> **Digital logic**

- **Represent numbers as 1s and 0s (binary)**
- **If you know circuits, ground represents 0, a nominal voltage represents 1**
- **Machine code is in binary, this enables us to make hardware for it**

Logic Gates

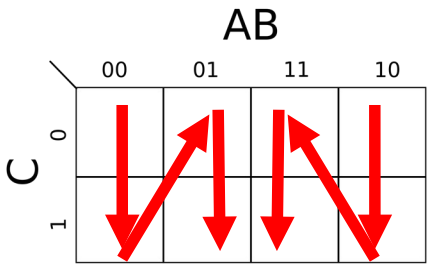
> Turn 1 and 0 inputs into outputs
  - For our sake, gates can only have up to four inputs
> Simplified ‘symbol’ view of circuits
> Quickly convey logic graphically

LOGIC FUNCTION	LOGIC SYMBOL	BOOLEAN EXPRESSION	TRUTH TABLE		
			INPUTS		OUTPUTS
			B	A	Y
AND		A+B=Y	0	0	0
			0	1	0
			1	0	0
			1	1	1
OR		A+B=Y	0	0	0
			0	1	1
			1	0	1
			1	1	1
inverter		$A=\overline{A}$		0	1
				1	0
NAND		$\overline{A+B}=Y$		0	1
			0	1	1
			0	0	1
			1	1	0
NOR		$\overline{A+B}=Y$	1	0	1
			0	1	0
			0	0	0
			1	1	0
XOR		$A \oplus B=Y$	1	0	0
			0	1	0
			0	0	1
			1	1	1
XNOR		$\overline{A \oplus B}=Y$	1	0	0
			0	0	1
			0	1	0
			1	0	0
			1	1	1

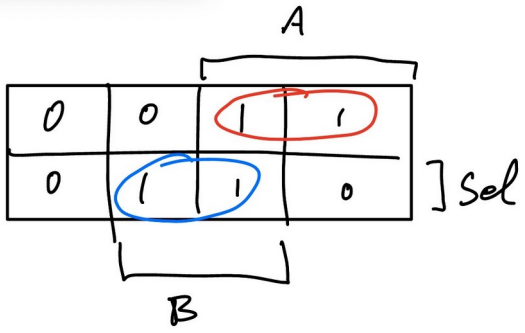
Logic Gates Example

"I want a circuit that outputs input A when sel is off, and B when input sel is on"

A	B	Sel	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



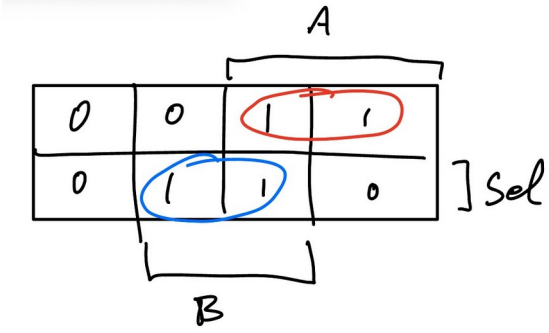
Karnaugh Map (K map)



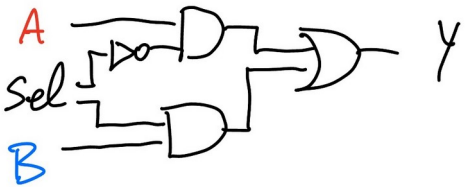
Logic Gates Example

"I want a circuit that outputs input A when sel is off, and B when input sel is on"

A	B	Sel	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



$$Y = A\overline{Sel} + B\,Sel$$



Hardware Description Language

Write code, compiler automatically lays the gates out

```
`timescale 1 ns /10 ps

module test (input logic   a,
             input logic   b,
             output logic  sel,
             output logic  out);

   // declare "variables" to write to
   logic not_sel, a_result, b_result;

   // invert using a gate

   not not_gate (not_sel, sel); // output always comes first, this is common assembly convention

   and and_gate_a (a_result, a, not_sel); // logical AND of a and not_sel
   and and_gate_b (b_result, b, sel); // logical AND of b and sel
or ~~and~~ result (out, a_result, b_result); // outputs result

endmodule
```

Hardware Description Language

There are better ways too (RTL), we might use to simplify things

```
`timescale 1 ns /10 ps

module test (input logic   a,
             input logic   b,
             output logic  sel,
             output logic  out);

   // This can also just be written in one line lmao
   assign out = ((a & ~sel) | (b & sel));

endmodule
```

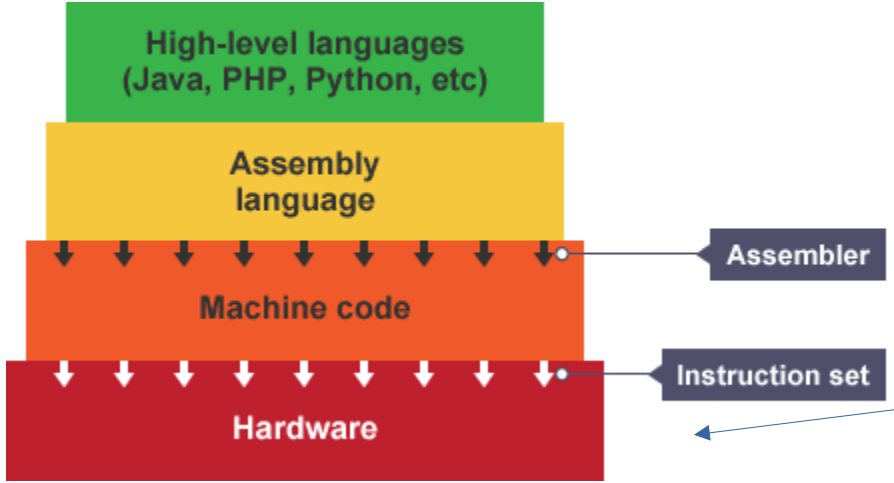
```
`timescale 1 ns /10 ps

module test (input logic   a,
             input logic   b,
             output logic  sel,
             output logic  out);

   // or like this
   assign out = sel ? a : b;

endmodule
```

Why does this matter for computers



We are here, making the hardware to interpret the machine code

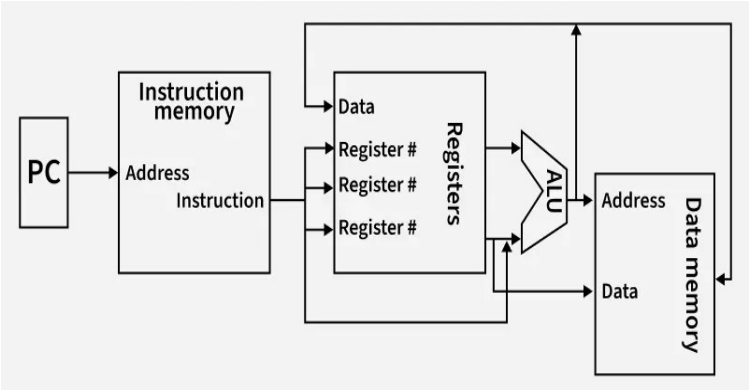
The Instruction Cycle

A processor follows these steps to execute instructions, we need to make the parts to enable this

1) **Fetch Instruction**
2) **Decode Instruction**
3) **Execute**
4) **Load from Memory**
5) **Write back**

The Data Path

Each of these parts is something we can make using our digital logic skills, so deliverables over time will be:

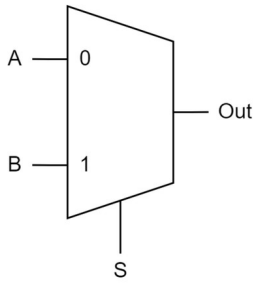


(Memory architecture would be an endeavor in it's own right, we'll ignore that for now)

- > **Large Multiplexer/decoder**
- > **Registers**
- > **ALU**
- > **Control Unit**
- > **Program Counter**

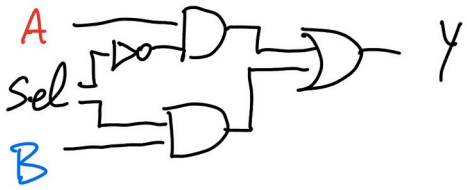
What you will do

Our current focus is going to be on producing a large multiplexor with more inputs, from now on, the left will represent what we've already established



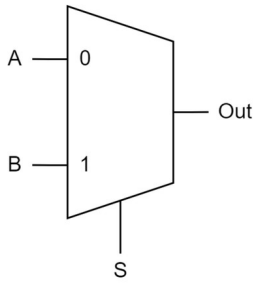
S	A	B	Out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

=



What you will do

Let’s try to make one with four inputs, and two selection lines. Using 2:1 multiplexers, make a 4:1 multiplexer with inputs A, B, C, D, and sel0, and sel1



S	A	B	Out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

What you will do

What about 16 inputs to 1 output? How many selection lines would we need?

> **Number of selection lines is always equal to $\log_2(n)$, where n is the number of inputs**
  - **4 sel lines**

Next time:

The muxes we made today are going to be used all over, but these specifically are a precursor to what's called the *registers* of a processor

> **What is a register?**

> **What does this have to do with multiplexers?**

> **Implementing the muxes**

> **Decoders**

> **Binary arithmetic**

Thanks for coming!

I’m going to try to structure these meetings in a way where we will learn something every time, and work on something every time.

Please feel free to DM me if you have any questions, comments, suggestions and I’ll do my best to answer